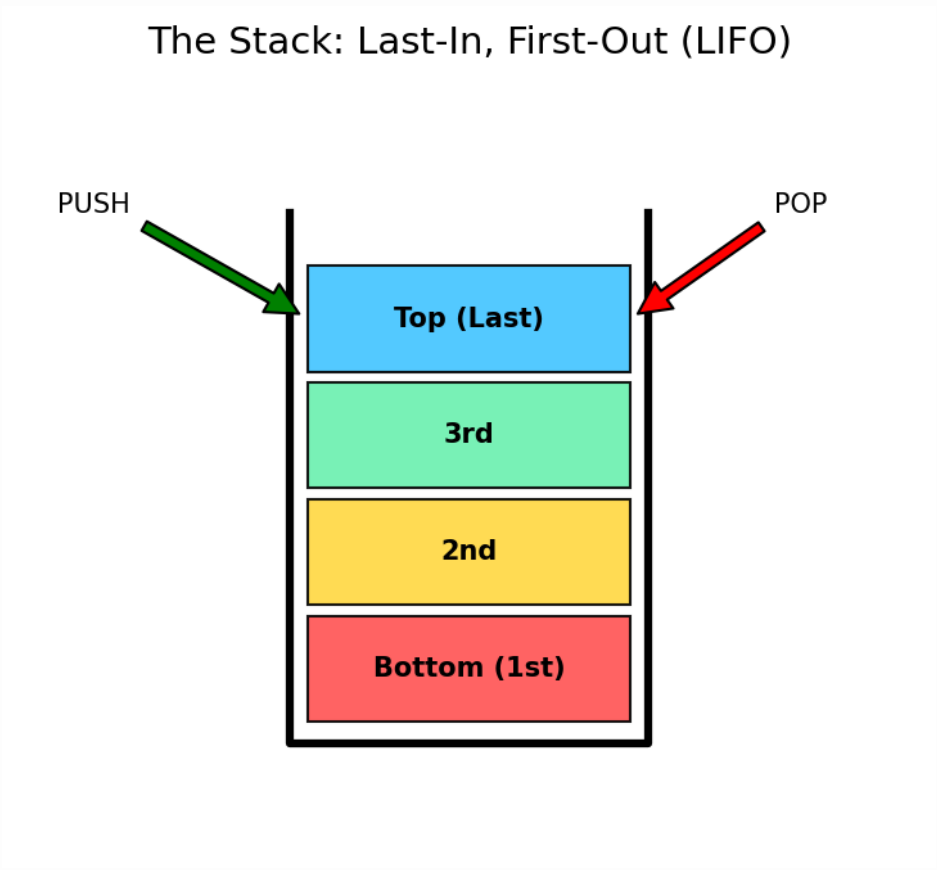
Stacks in Python

The LIFO Principle & Technical Implementation

1. The LIFO Concept

A **Stack** is a linear data structure that follows the **Last-In, First-Out (LIFO)** principle. Think of it as a stack of cafeteria trays.

PUSH: Add to top → **POP:** Remove from top



Core Operations

- `push(item)`: Adds to the top O(1)
- `pop()`: Removes the top item O(1)
- `peek()`: Look at top item O(1)

2. Python Syntax & Implementation

Using Python's `list` to implement a stack efficiently:

```
class Stack:
    def __init__(self):
        self.items = [] # Underlying storage

    def push(self, item):
        # .append() adds to the end (The Top)
        self.items.append(item)

    def pop(self):
        # .pop() with no arguments removes the last element
        if not self.is_empty():
            return self.items.pop()

    def peek(self):
        # Negative indexing [-1] gets the top/last element
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0
```

3. Problem 1: Valid Parentheses

Goal: Check if brackets `()`, `[]`, `{}` are nested and closed correctly.

```python
def isValid(s: str) -> bool:
    stack = []
    # Map closing to opening for easy lookup
    lookup = {')': '(', '}': '{', ']': '['}

    for char in s:
        if char in lookup:
            # It's a closing bracket. Pop from stack.
            top = stack.pop() if stack else '#'
            if lookup[char] != top:
                return False
        else:
            # It's an opening bracket. Push to stack.
            stack.append(char)

    return not stack # Stack must be empty if valid
```

4. Problem 2: Queue using Stacks

Goal: Use two LIFO stacks to mimic FIFO (First-In, First-Out).

```
class MyQueue:
    def __init__(self):
        self.s1 = [] # Input Stack
        self.s2 = [] # Output Stack

    def push(self, x: int):
        self.s1.append(x)

    def pop(self) -> int:
        self.move_if_needed()
        return self.s2.pop()

    def move_if_needed(self):
        # Reverse order by dumping s1 into s2
        if not self.s2:
            while self.s1:
                self.s2.append(self.s1.pop())
```